

GymFlow Roles and Permissions Report

Overview

GymFlow is a multi-tenant gym management SaaS designed for gym owners and managers to handle customers, staff, equipment, attendance, classes, memberships, inventory, and related operations in one place.[cite:1] The planned access model uses a layered hierarchy with the main application owner at the top, followed by gym managers, and then staff and trainers within each gym tenant.[cite:1]

A strong role design is critical because role-based access control works best when application roles are clearly defined, permissions are grouped by responsibility, and users receive only the minimum access needed to do their jobs.[cite:84] [cite:86] [cite:56] Multi-tenant SaaS systems also require strict tenant isolation so that users from one gym cannot access the data of another gym, even if both gyms are stored in the same system.[cite:68] [cite:94]

Access model

The recommended GymFlow model has two layers of authority: a platform layer and a gym tenant layer.[cite:1] [cite:68] The platform layer belongs to the software owner and controls the overall SaaS environment, while the tenant layer belongs to each gym and handles day-to-day operations inside that gym.[cite:68] [cite:84]

This structure follows common RBAC guidance that recommends defining app roles first, creating a clear role hierarchy, and applying least-privilege access rather than granting permissions directly to individual users wherever possible.[cite:84] [cite:56] [cite:90] It also matches GymFlow’s manager-focused product direction, where the main users are gym operators rather than individual gym members.[cite:51] [cite:1]

Role hierarchy

The proposed role hierarchy is shown below.

Layer	Role	Scope	Purpose
Platform	Super Admin	Entire SaaS platform	Owns and operates GymFlow, manages tenants, subscriptions, support access, and platform-wide settings.[cite:1] [cite:84]
Tenant	Gym Manager	One gym tenant	Runs one gym’s operations, staff, memberships, billing, reports, and settings.[cite:1] [cite:51]
Tenant	Staff	One gym tenant	Handles front-desk and operational work such as check-ins, member updates, class bookings, and payment entry.[cite:51] [cite:72]
Tenant	Trainer	One gym tenant	Handles classes, personal training sessions, attendance marking, and member training notes within assigned scope.[cite:51] [cite:72]

This four-role structure is intentionally simple for version one because RBAC systems become harder to manage when too many overlapping roles are introduced too early.[cite:56] [cite:86] It gives enough separation for security and usability without creating unnecessary complexity in the database, middleware, or user interface.[cite:56] [cite:90]

Super Admin

Purpose

The Super Admin is the platform owner role. This role exists above all gyms and is responsible for the GymFlow product as a SaaS platform rather than for day-to-day work in any single gym.[cite:1] [cite:84] In practical terms, this is the role used by the main developer or business owner to create and manage gym tenants, monitor subscriptions, provide support, and maintain platform-level settings.[cite:1] [cite:68]

Core responsibilities

The Super Admin should be able to:

- Create, activate, suspend, and delete gym tenants when business rules permit.[cite:84] [cite:68]
- Manage plan limits, subscription status, billing metadata, and tenant onboarding workflows.[cite:84]
- View platform-wide analytics such as total gyms, active subscriptions, total members across tenants, and system usage trends.[cite:84]
- Access support tools such as tenant diagnostics, audit logs, and controlled impersonation or support sessions when necessary.[cite:56] [cite:68]
- Configure platform-level defaults, feature flags, integration availability, and global notification templates if the product supports them.[cite:84] [cite:86]

Permissions

The Super Admin should have full permission over platform modules, but those permissions should still be explicitly logged and monitored because high-privilege roles carry the highest security risk.[cite:56] [cite:88] Least-privilege principles still apply here, meaning even platform actions should be structured, auditable, and revocable.[cite:90] [cite:93]

What Super Admin should not do routinely

Although this role can technically access tenant data for support purposes, it should not be used for routine gym operations such as marking attendance or collecting payments.[cite:56] [cite:68] A clean separation between platform administration and tenant operations keeps the system safer, easier to audit, and more professional.[cite:56] [cite:94]

Gym Manager

Purpose

The Gym Manager is the highest operational role inside a single gym tenant. This role is intended for the gym owner, branch manager, or senior operator who runs the gym's daily business activities.[cite:1] [cite:51]

Core responsibilities

The Gym Manager should be able to:

- Manage member records, membership plans, renewals, freezes, and expiries.[cite:1] [cite:72]
- Oversee attendance policies, class schedules, trainer assignments, and staff operations. [cite:1] [cite:51]
- Handle billing workflows such as invoices, dues, refunds, discounts, package sales, and expense tracking.[cite:72] [cite:91]
- Manage inventory, equipment tracking, low-stock alerts, and operational settings within the gym.[cite:1] [cite:72]
- Create and manage tenant-level users such as staff and trainers, including role assignments inside that gym.[cite:1] [cite:84]
- View full tenant reports covering revenue, attendance, active members, expired plans, classes, and staff performance.[cite:72]

Permissions

The Gym Manager should have full control within the boundaries of the tenant to which the user belongs.[cite:68] [cite:94] This means a manager for Gym A can fully manage Gym A but can never see Gym B, even if both are customers of the same SaaS platform.[cite:68]

The role should typically include create, read, update, export, and limited delete rights across most gym modules.[cite:86] [cite:91] For destructive actions like deleting members, changing billing rules, or modifying role assignments, soft-delete patterns, confirmation flows, and audit logs should be used to reduce operational mistakes.[cite:56]

Management authority

Gym Managers should be allowed to create Staff and Trainer accounts, reset their access, and deactivate users who leave the organization.[cite:56] [cite:84] This aligns with RBAC guidance that emphasizes defined role ownership, regular review of assignments, and prompt removal of permissions when responsibilities change.[cite:56]

Staff

Purpose

The Staff role is for front-desk personnel, receptionists, and operational assistants who keep the gym running day to day.[cite:51] This role should be optimized for speed and workflow efficiency rather than strategic control.[cite:51]

Core responsibilities

The Staff role should be able to:

- Search, create, and update member records within allowed fields.[cite:72]
- Mark member attendance, fix check-in issues, and manage front-desk check-in workflows.[cite:1] [cite:72]
- Assign or renew membership plans if the gym's policy allows front-desk staff to do so. [cite:51]
- Record payments, generate receipts, and track pending dues, while leaving refund approval and financial configuration to the manager.[cite:91] [cite:56]
- Book classes, reschedule sessions, manage basic leads, and update limited inventory transactions such as stock-out for product sales.[cite:72]
- Send predefined reminders such as renewal messages or due-payment notices if the gym enables notification workflows.[cite:55] [cite:72]

Permissions

The Staff role should receive only operational permissions and should not receive strategic or security-sensitive permissions.[cite:56] [cite:90] In practice, Staff users should normally not be allowed to change user roles, update system settings, approve refunds, view complete financial analytics, or access data for other gyms.[cite:56] [cite:68]

Operational design notes

Because Staff users perform repetitive tasks, their dashboard should focus on quick actions such as member search, attendance marking, payment collection, plan renewal, and today's bookings.[cite:51] [cite:72] A streamlined UI for this role reduces training time and improves front-desk throughput during busy hours.[cite:51]

Trainer

Purpose

The Trainer role is for coaches and instructors whose work centers on classes, personal training sessions, and member engagement rather than full business administration.[cite:51] This role should be narrow by default because trainers rarely need access to broad financial, inventory, or administrative functions.[cite:56] [cite:90]

Core responsibilities

The Trainer role should be able to:

- View assigned classes, sessions, and schedules.[cite:72]
- Mark attendance for classes or PT sessions within their assignment scope.[cite:72]
- View assigned members or class participants when required for coaching operations.
[cite:51]
- Add training notes, progress comments, session remarks, or internal observations where the product supports them.[cite:51]
- View limited reports such as attendance for their sessions, class occupancy, or assigned member participation if those metrics are useful.[cite:72]

Permissions

The Trainer role should not be allowed to manage billing, tenant settings, broad member records, or role assignment workflows.[cite:56] [cite:90] If compensation tracking or PT commissions are later added, trainers may be allowed to view their own earnings summary only, not organization-wide payroll data.[cite:56]

Operational design notes

The Trainer interface should emphasize schedule clarity, class attendance marking, and notes rather than admin-heavy tables.[cite:51] This keeps the experience aligned with their job and avoids giving access to modules they do not need.[cite:56]

Module-by-module permissions

The following table summarizes recommended access by module.

Module	Super Admin	Gym Manager	Staff	Trainer
Tenant management	Full access.[cite:84] [cite:68]	No access.	No access.	No access.
Gym settings	Support/global controls. [cite:84]	Full tenant control. [cite:72]	Profile-only or none.	Profile-only or none.
Members	Support access when needed.[cite:68]	Full control. [cite:1]	Create/read/update limited operational access.[cite:72]	View assigned members only. [cite:51]
Memberships	Support access when needed.[cite:68]	Full control. [cite:1]	Assign/renew if allowed.[cite:51]	View-only where relevant.
Attendance	Support visibility. [cite:68]	Full control. [cite:1]	Full operational attendance handling. [cite:72]	Attendance for assigned classes/sessions. [cite:72]

Module	Super Admin	Gym Manager	Staff	Trainer
Classes	Support visibility. [cite:68]	Full control. [cite:72]	Booking and schedule support.[cite:72]	Assigned class management only. [cite:72]
Billing	Subscription/platform-level context.[cite:84]	Full tenant financial control. [cite:91]	Payment entry and receipts only.[cite:91]	No access.
Staff users	Support/global visibility. [cite:84]	Full tenant user control. [cite:84]	No management authority.	No management authority.
Inventory	Support visibility. [cite:68]	Full control. [cite:72]	Limited transaction updates.[cite:72]	View/request only.
Reports	Platform analytics. [cite:84]	Full gym reports. [cite:72]	Limited operations reports.[cite:72]	Limited schedule/class reports.[cite:72]
Notifications	Global/default support. [cite:84]	Full tenant campaign control. [cite:55]	Predefined operational sends.[cite:55]	Very limited or none.
Audit logs	Full visibility.[cite:56]	Tenant-level visibility. [cite:56]	None or very limited.	None.

Recommended restrictions

Several restrictions should be built in from the start.

- Every tenant-scoped table should include a tenant identifier so access can always be filtered by gym.[cite:68][cite:94]
- High-risk actions such as role changes, refunds, settings updates, and deletes should require stronger permissions and audit logging.[cite:56][cite:90]
- Deactivation should usually be preferred over hard deletion for users and records that affect billing, attendance, or history.[cite:56]
- Permission changes should be reviewed periodically, especially when staff leave or responsibilities change.[cite:56]
- Support access by the Super Admin should be transparent and logged to maintain trust with gym customers.[cite:56][cite:88]

Permission design recommendations

GymFlow should not rely on role names alone in application logic. Instead, roles should map to granular permission keys such as `member.create`, `member.update`, `billing.collect`, `refund.approve`, `attendance.mark`, and `settings.branding.update`. [cite:86][cite:84] This

approach makes the backend easier to maintain, reduces authorization bugs, and gives room for future expansion such as custom roles or branch-specific variations.[cite:86][cite:94]

For implementation, each request should be validated in three steps: user identity, tenant scope, and action permission.[cite:68][cite:84] That means GymFlow should first confirm who the user is, then confirm which tenant the user belongs to, and finally confirm whether that role has the requested permission inside that tenant.[cite:68][cite:94]

Recommended future expansion

The four-role structure is appropriate for version one, but the permission system should be designed so that custom roles can be added later without rewriting the entire application.[cite:56][cite:86] Future tenant roles may include accountant, branch manager, nutritionist, support executive, or read-only auditor, but those should only be introduced after real customer demand appears.[cite:56]

In the same way, GymFlow can later expand from pure RBAC toward hybrid authorization models that combine role checks with attributes such as branch, assigned class, or resource ownership when the product becomes more advanced.[cite:86][cite:94] For the current stage, however, tenant-aware RBAC with least privilege is the most practical and stable foundation.[cite:68][cite:90]

Final recommendation

The best role structure for GymFlow is a two-layer access model made up of one platform role and three tenant roles: Super Admin, Gym Manager, Staff, and Trainer.[cite:1][cite:68] This structure reflects the product's business reality, keeps authorization understandable, and aligns with established RBAC best practices such as clear hierarchy, least privilege, auditability, and tenant isolation.[cite:56][cite:84][cite:90]

A well-designed role system will make GymFlow safer, easier to maintain, and easier to sell to gym owners because it shows clear control over who can view, update, and manage critical business data.[cite:1][cite:56] It also creates a strong base for future modules such as multi-branch support, custom roles, advanced reporting, or automated workflows without needing to redesign the security model later.[cite:33][cite:68]